How Java Runs: Compilation, Bytecode and the JIT

12 min read Updated 28 Jul 2026 Topic 2 of 65

Bookmark Mark done

Why it matters

"Is Java compiled or interpreted?" is asked constantly, and the confident one-word answers are both wrong. Java is compiled *and* interpreted *and* compiled again — and the reason it works that way is the single most interesting engineering decision in the platform. Understanding it explains a long list of things that otherwise look like folklore: why a Java service is slow for its first thirty seconds and fast afterwards, why benchmarks need warm-up, why the same code can be faster than equivalent C++ in a long-running server and slower in a command-line script.

It also underpins every performance conversation you will have. When Chapter 9 discusses tiered compilation and Chapter 8 discusses why a `volatile` read cannot be optimised away, both are talking about the machinery introduced here.

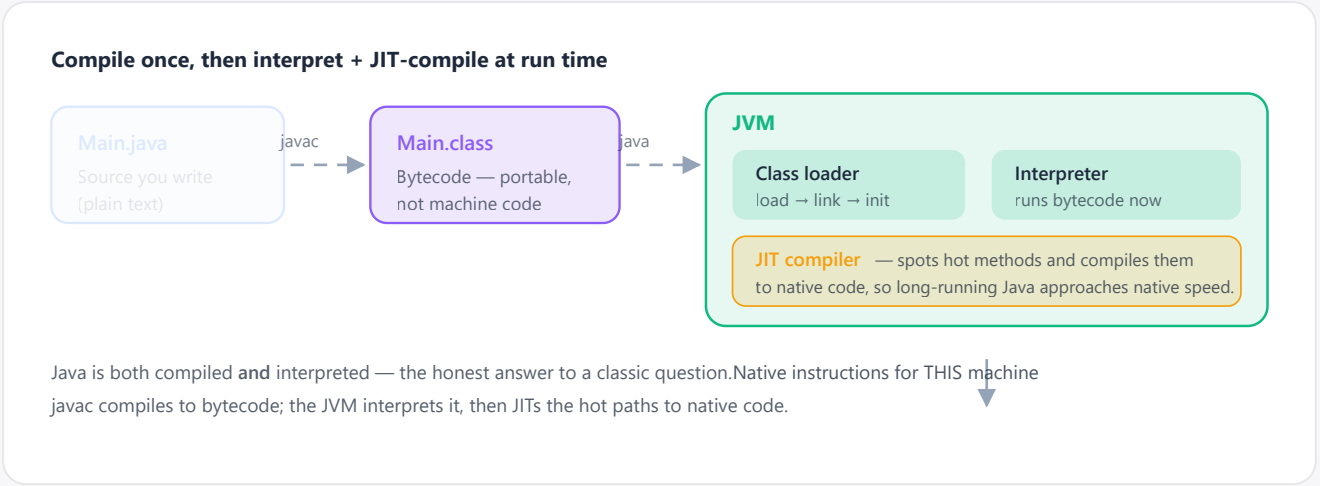
The concept, simply

Java takes a middle path between the two classic models. A traditional compiled language (C) turns source into native machine code once, ahead of time — fast, but the binary only runs on the platform it was built for. A traditional interpreted language (classic Python) reads and executes source line by line every run — portable, but slow.

Java splits the difference in two steps:

1. **Ahead of time, `javac` compiles your source into *bytecode*** — instructions for the abstract JVM, not for any real CPU. This step does the type checking and most of the syntactic heavy lifting, once.

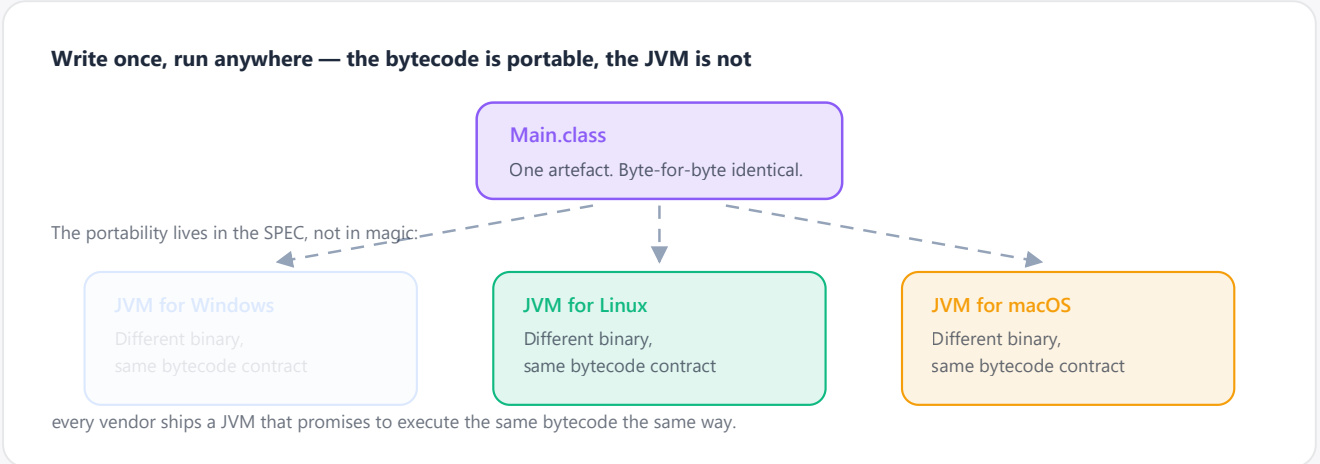
into real native machine code, which then runs directly on the CPU.



Two compilations, not one. javac makes portable bytecode ahead of time; the JIT makes native code at run time, for the methods that actually matter.

So the honest answer to the interview question is: **both**. Compiled to bytecode ahead of time, then interpreted and selectively compiled to native code at run time.

Splitting it this way is also what buys portability. Because step 1 targets the abstract JVM rather than a real CPU, the artefact it produces is the same everywhere; only step 2 has to know what machine it is on.



"Write once, run anywhere" is not magic — it is a specification. The bytecode is identical everywhere; the JVM that consumes it is a different native binary on every platform.

How it works

desugaring) and emits one `.class` file per class or nested class. Notably, `javac` optimises very little. It is not trying to be clever — it deliberately leaves optimisation to the JIT, which will know things `javac` cannot possibly know, like which branch is taken 99% of the time in production.

Step 2 — class loading. When you run `java Hello`, the JVM starts, and the class loader finds `Hello.class`, loads it, links it (verify, prepare, resolve) and initialises it. Chapter 9 covers this in detail; for now the point is that loading is *lazy* — a class is loaded the first time it is actually needed, not up front.

Step 3 — interpretation. The JVM begins executing bytecode with an interpreter. Starting this way is a deliberate choice: interpretation begins *instantly*, whereas compiling everything to native code first would delay startup enormously for code that may run once.

Step 4 — profiling and JIT compilation. While interpreting, the JVM counts. How many times has this method been invoked? How many times has this loop iterated? Once a counter crosses a threshold the method is deemed *hot* and queued for compilation on a background thread. HotSpot uses **tiered compilation**: a fast, lightly-optimising compiler (**C1**) kicks in early to get code running natively soon, while a slower, aggressively-optimising compiler (**C2**) recompiles the hottest methods later with far better output. Compilation happens on background threads, so the application keeps running while it waits.

What makes the JIT genuinely powerful is that it optimises using *runtime facts* that no ahead-of-time compiler can have:

Optimisation	What it does
Inlining	Replaces a call to a small hot method with its body — the single highest-value c
Monomorphic dispatch	Observes that a virtual call site has only ever seen one implementing class, and

Dead code elimination	Removes branches that profiling shows are never taken
Loop unrolling / hoisting	Reduces per-iteration overhead; lifts invariant work out of loops
Escape analysis	Proves an object never leaves a method, then removes the allocation entirely (s

Because these decisions rest on observed behaviour, they can turn out to be wrong — a call site that saw one class for a million calls suddenly sees a second. The JVM handles this with **deoptimisation**: the compiled code contains guards, and when a guard fails, execution falls back to the interpreter and the method is recompiled with the new information. This is why Java can perform speculative optimisations that a static compiler would never dare attempt.

Code examples

Bytecode is not a mystery; you can read it. `javap` ships with the JDK.

JAVA Copy

```java
public class Add {
    public static int sum(int a, int b) {
        return a + b;
    }
}
```

BASH Copy

```bash
javac Add.java
javap -c Add.class
```

```
public static int sum(int, int);
  Code:
     0: iload_0        // push local variable 0 (a) onto the operand stack
     1: iload_1        // push local variable 1 (b)
     2: iadd           // pop two ints, add, push the result
     3: ireturn        // pop the int and return it
```

Notice there are no registers — only pushes and pops against an operand stack. That is what makes bytecode portable: it assumes nothing about the CPU it will eventually run on.

You can also watch the JIT work. This prints every method as it is compiled:

BASH Copy

```bash
# Which methods got compiled, at which tier, and when.
java -XX:+PrintCompilation Main

# Disable the JIT entirely — everything stays interpreted. A dramatic way
# see how much of Java's speed comes from the JIT rather than from javac.
java -Xint Main

# Force everything through the fast-but-shallow C1 compiler only.
java -XX:TieredStopAtLevel=1 Main   # what Spring Boot devtools uses for q
```

The warm-up effect is easy to observe, and explains why naive benchmarks lie:

JAVA Copy

```java
public class Warmup {
    static long work() {
        long total = 0;
        for (int i = 0; i < 10_000_000; i++) total += i % 7;
        return total;
    }
```

```
            work();
            System.out.printf("run %d: %d ms%n", run, (System.nanoTime() -
        }
    }
}
// Typical shape: run 1 is several times slower than run 5. Nothing change
// the code — the JIT compiled work() to native somewhere around run 2.
// This is exactly why real measurement uses JMH, which warms up first.
```

Under the hood

Two nuances separate a good answer from a great one.

AOT compilation exists too. The interpret-then-JIT model is optimised for long-running server processes, where a few seconds of warm-up buys hours of peak performance. It is a poor fit for short-lived processes — a CLI tool, or a serverless function that starts, handles one request and dies, never getting warm. **GraalVM Native Image** addresses this by compiling Java ahead of time into a standalone native executable: startup drops from hundreds of milliseconds to a few, and memory use falls sharply. The trade-offs are real — no JIT means no profile-guided peak performance, and reflection or dynamic proxies must be declared at build time, which is why frameworks needed explicit support (Spring Boot 3's AOT processing, Quarkus, Micronaut). Knowing when the JIT model is *wrong* is a genuinely senior signal.

Escape analysis undermines a rule you were taught. "Objects always live on the heap" is the model, and it is what you should say. But if the JIT can prove an object never escapes its method — never stored in a field, never returned, never passed somewhere that keeps it — it may eliminate the allocation altogether and keep the fields in registers or on the stack. This is *scalar replacement*. It does not change how you write code, and you cannot rely on it, but it is why "allocation is expensive, avoid objects in loops" is much weaker advice on the JVM than it sounds.

One more practical note: **the code cache**. JIT-compiled native code lives in a fixed-size region of native memory. On very large applications it can fill, at which point the JVM

diagnose without knowing the JIT exists.

Common mistakes

- **Answering "compiled" or "interpreted" flatly.** Both are wrong on their own. Say both, and explain the two stages.
- **Thinking `javac` optimises heavily.** It barely optimises at all — by design. The JIT does that work with better information.
- **Benchmarking without warm-up.** Timing the first iteration measures the interpreter, not your code. Use JMH.
- **Believing bytecode is machine code.** Bytecode targets the abstract JVM. Machine code targets a real CPU. The JIT is the bridge.
- **Assuming the JIT compiles everything.** It compiles what is *hot*. Cold code stays interpreted forever, and that is the correct behaviour.
- **Expecting GraalVM Native Image to be strictly faster.** It starts far faster and uses less memory, but peak throughput is often *lower* — there is no profile-guided JIT.

Interview lens

> **Q: "Is Java compiled or interpreted?"**
> A: Both. `javac` compiles source to platform-independent bytecode ahead of time. At run time the JVM interprets that bytecode and profiles it, then the JIT compiles the hot methods to native machine code. So it is compiled twice, with an interpreter in between.

> **Q: "What is the JIT compiler, and why is it worth the complexity?"**
> A: It compiles frequently-executed bytecode to native code at run time. It is worth it because it optimises with information a static compiler cannot have — actual branch frequencies, actual receiver types at a call site. That enables aggressive

inlining and speculative optimisations, guarded so that if the assumption breaks the JVM deoptimises back to the interpreter and recompiles.

Q: "Why is my Java application slow for the first few seconds?"

A: Warm-up. Classes are being loaded lazily, and the code is still interpreted while the JVM gathers profiling data. Once methods cross their compilation thresholds the JIT produces native code and throughput climbs. It is also why load tests must warm up before measuring, and why short-lived processes get none of the benefit.

Q: "What is tiered compilation?"

A: HotSpot's strategy of using two JIT compilers. C1 compiles early with light optimisation so code goes native quickly; C2 recompiles the hottest methods later with heavy optimisation. It gets you most of the responsiveness of C1 and most of the peak throughput of C2. `-XX:TieredStopAtLevel=1` restricts to C1, which is a common trick for fast developer restarts.

Q: "What is deoptimisation?"

A: The JIT makes speculative optimisations based on observed behaviour — for example, compiling a virtual call as a direct call because only one type has ever been seen. The compiled code keeps a guard. If the guard fails, the JVM discards that compiled version, resumes in the interpreter, and recompiles with the corrected profile. It is what makes speculation safe.

Q: "How would you make a Java service start faster?"

A: Depends what dominates. If it is JIT warm-up and the process is short-lived, GraalVM Native Image is the structural answer — AOT-compile to a native binary, trading peak throughput for startup and memory. Short of that: `-XX:TieredStopAtLevel=1` to skip C2, class-data sharing (CDS) to cut class-loading cost, trimming the classpath, and deferring eager initialisation so less work happens before the first request.

A: It disassembles a `.class` file. `javap -c` prints the bytecode, which is how you settle arguments about what the compiler actually generated — string concatenation in a loop, whether a lambda became an `invokedynamic`, where autoboxing was inserted. It is the fastest way to prove what the compiler did rather than guess.

Cheat-card

- `javac` → **bytecode** (portable, ahead of time). JVM → **interpret + JIT** → native.
- Answer to "compiled or interpreted?" is **both**.
- Bytecode is **stack-based**, no registers — that is what makes it portable. Read it with `javap -c`.
- **Tiered compilation**: C1 fast/shallow early, C2 slow/deep for the hottest code.
- JIT wins come from **runtime profile data**: inlining, monomorphic dispatch, dead-code elimination, escape analysis.
- **Deoptimisation** = guard fails → back to interpreter → recompile. Makes speculation safe.
- **Warm-up is real** — benchmark with JMH, never with a stopwatch on run 1.
- **GraalVM Native Image** = AOT: fast start, low memory, lower peak throughput, reflection must be declared.
- `-Xint` disables the JIT; `-XX:+PrintCompilation` shows it working.

QUICK CHECK — no marks, just for you.

1. Is Java compiled or interpreted?

- ◯ Both: javac compiles to bytecode ahead of time, then the JVM interprets it and JIT-compiles hot methods to native code
- ◯ Compiled — javac produces native machine code for the target platform

- ◯ Interpreted — the JVM reads the .java source line by line

Check answer

2. Why can the JIT compiler produce optimisations that an ahead-of-time compiler cannot?

- ◯ It targets bytecode rather than machine code, which is easier to optimise
- ◯ It observes real runtime behaviour — actual branch frequencies and receiver types — and can speculate, with guards that deoptimise if the assumption breaks
- ◯ It has more time available, because it runs while the program is idle
- ◯ It can rewrite the original source code before compiling it

Check answer

3. A load test shows the service is much slower during its first few seconds. What is the standard explanation?

- ◯ The JVM verifies every class on the classpath before accepting the first request
- ◯ Bytecode is decompressed on first use and cached afterwards
- ◯ Warm-up: classes are still being loaded lazily and code is still interpreted until the JIT compiles the hot methods
- ◯ The garbage collector always runs a full collection at startup

Check answer

Prev
JDK, JRE and JVM
Next
Primitives vs References

LEARN MORE

ARTICLES & DOCS

- The class file format (JVMS §4)
  docs.oracle.com
- javap — the class file disassembler
  docs.oracle.com
- GraalVM Native Image reference manual
  graalvm.org
- JMH — the Java Microbenchmark Harness
  github.com

VIDEOS



How Java Works: Understanding JVM, JRE, and JDK | Java Architecture | Java Tutorial For Beginners
Talented Developer



Java 8 JVM Architecture Explained 🔥 | Heap, Stack, Metaspace (PermGen?) & JVM Memory | Part 2
Vaibhav Code Lab

Hand-picked references for this lesson.